



Parallel & Distributed Computing

Lecture week 11:

Distributed Memory Programming

Farhad M. Riaz

Farhad.Muhammad@numl.edu.pk

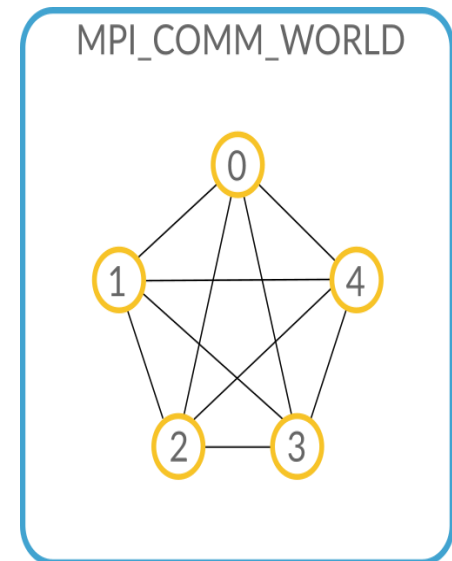
Department of Computer Science
NUML, Islamabad

Include files

- The MPI include file
 - `mpi.h`
- Defines many constants used within MPI programs
- In C defines the interfaces for the functions
- Compilers know where to find the include files

Communicators

- A parameter for most MPI calls
- A collection of processors working on some part of a parallel job
- `MPI_COMM_WORLD` is defined in the MPI include file as all of the processors in your job
- Can create subsets of `MPI_COMM_WORLD`
- Processors within a communicator are assigned numbers 0 to $n-1$



Data types

- When sending a message, it is given a data type
- Predefined types correspond to "normal" types
 - MPI_REAL, MPI_FLOAT - Fortran real and C float respectively
 - MPI_DOUBLE_PRECISION, MPI_DOUBLE - Fortran double precision and C double respectively
 - MPI_INTEGER and MPI_INT - Fortran and C integer respectively
- User can also create user-defined types

Minimal MPI program

```
#include <mpi.h> /* the mpi include file */

/* Initialize MPI */
ierr=MPI_Init(&argc, &argv);

/* How many total PEs are there */
ierr=MPI_Comm_size(MPI_COMM_WORLD, &nPEs);

/* What node am I (what is my rank? */
ierr=MPI_Comm_rank(MPI_COMM_WORLD, &iam);
...
ierr=MPI_Finalize();
```

MPI “Hello, World”

- A parallel hello world program
 - Initialize MPI
 - Have each node print out its node number
 - Quit MPI

```
1  #include <stdio.h>
2  #include <mpi.h>
3
4  int main(int argc, char** argv){
5      int process_Rank, size_Of_Cluster;
6
7      MPI_Init(&argc, &argv);
8      MPI_Comm_size(MPI_COMM_WORLD, &size_Of_Cluster);
9      MPI_Comm_rank(MPI_COMM_WORLD, &process_Rank);
10
11      printf("Hello World from process %d of %d\n", process_Rank, size_Of_Cluster);
12
13      MPI_Finalize();
14      return 0;
15 }
```

Hello World from process 3 of 4
Hello World from process 2 of 4
Hello World from process 1 of 4
Hello World from process 0 of 4

C/MPI version of “Hello, World”

```
#include <stdio.h>
#include <math.h>
#include <mpi.h>
int main(int argc, char *argv[ ])
{
    int myid, numprocs;

    MPI_Init (&argc, &argv) ;
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs) ;
    MPI_Comm_rank(MPI_COMM_WORLD, &myid) ;
    printf("Hello from %d\n", myid) ;
    printf("Numprocs is %d\n", numprocs) ;

    MPI_Finalize():
}
```


Basic Communications in MPI

- Data values are transferred from one processor to another
 - One process sends the data
 - Another receives the data
- Synchronous
 - Call does not return until the message is sent or received
- Asynchronous
 - Call indicates a start of send or receive operation, and another call is made to determine if call has finished

Synchronous Send

- MPI_Send: Sends data to another processor
- Use MPI_Receive to "get" the data

```
MPI_Send(&buffer, count, datatype,  
destination, tag, communicator);
```

- Call blocks until message on the way

```
void* message;           //Address for the message you are sending.  
int count;               //Number of elements being sent through the address  
MPI_Datatype datatype;  //The MPI specific data type being passed through  
int dest;                //Rank of destination process.  
int tag;                 //Message tag.  
MPI_Comm comm;           //The MPI Communicator handle.
```

```
void* message;           //Address to the message you are receiving.  
int count;               //Number of elements being sent through the address  
MPI_Datatype datatype;  //The MPI specific data type being passed through  
int from;                //Process rank of sending process.  
int tag;                 //Message tag.  
MPI_Comm comm;           //The MPI Communicator handle.  
MPI_Status* status;      //Status object.
```

MPI_Send

- Buffer: The data
- Count : Length of source array (in elements, 1 for scalars)
- Datatype : Type of data, for example
MPI_DOUBLE_PRECISION (fortran) ,
MPI_INT (C), etc
- Destination : Processor number of destination processor in communicator
- Tag : Message type (arbitrary integer)
- Communicator : Your set of processors
- Ierr : Error return (Fortran only)

Synchronous Receive

- Call blocks until message is in buffer

- **MPI_Recv(&buffer, count, datatype, source, tag, communicator, &status);**

- Status - contains information about incoming message

- **MPI_Status status;**

Status

- status is a structure of type MPI_Status which contains three fields MPI_SOURCE, MPI_TAG, and MPI_ERROR
- status.MPI_SOURCE, status.MPI_TAG, and status.MPI_ERROR contain the source, tag, and error code respectively of the received message

MPI_Recv

- Buffer: The data
- Count : Length of source array (in elements, 1 for scalars)
- Datatype : Type of data, for example :
MPI_DOUBLE_PRECISION, MPI_INT, etc
- Source : Processor number of source processor in communicator
- Tag : Message type (arbitrary integer)
- Communicator : Your set of processors
- Status: Information about message
- Ierr : Error return (Fortran only)

Basic MPI Send and Receive

- A parallel program to send & receive data
 - Initialize MPI
 - Have processor 0 send an integer to processor 1
 - Have processor 1 receive an integer from processor 0
 - Both processors print the data
 - Quit MPI

Simple Send & Receive Program

```
#include <stdio.h>
#include "mpi.h"
#include <math.h>
/*****
This is a simple send/receive program in MPI
*****/
int main(argc,argv)
int argc;
char *argv[];
{
    int myid, numprocs;
    int tag,source,destination,count;
    int buffer;
    MPI_Status status;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&myid);
```

Simple Send & Receive Program (cont.)

```
tag=1234;
source=0;
destination=1;
count=1;
if(myid == source) {
    buffer=5678;
    MPI_Send(&buffer,count,MPI_INT,destination,tag,MPI_COMM_WORLD);
    printf("processor %d sent %d\n",myid,buffer);
}
if(myid == destination) {
    MPI_Recv(&buffer,count,MPI_INT,source,tag,MPI_COMM_WORLD,&status);
    printf("processor %d got %d\n",myid,buffer);
}
MPI_Finalize();
}
```

```
#include <stdio.h>
#include <mpi.h>
int main(int argc, char** argv){
    int process_Rank, size_Of_Cluster, message_Item;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size_Of_Cluster);
    MPI_Comm_rank(MPI_COMM_WORLD, &process_Rank);

    if(process_Rank == 0){
        message_Item = 42;
        printf("Sending message containing: %d\n", message_Item)
    }
    else if(process_Rank == 1){
        printf("Received message containing: %d\n", message_Item)
    }
    MPI_Finalize();
    return 0;
}
```

The 6 Basic C MPI Calls

- MPI is used to create parallel programs based on message passing
- Usually the same program is run on multiple processors
- The 6 basic calls in C MPI are:
 - `MPI_Init( &argc, &argv)`
 - `MPI_Comm_rank( MPI_COMM_WORLD, &myid )`
 - `MPI_Comm_Size( MPI_COMM_WORLD, &numprocs)`
 - `MPI_Send(&buffer, count, MPI_INT, destination, tag, MPI_COMM_WORLD)`
 - `MPI_Recv(&buffer, count, MPI_INT, source, tag, MPI_COMM_WORLD, &status)`
 - `MPI_Finalize()`

Further Reading

- <https://computing.llnl.gov/tutorials/mpi/>

Communicators and Ranks

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
print('My rank is ',rank)
```

Point-to-Point Communication

```
from mpi4py import MPI
import numpy
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
if rank == 0:
    data = {'a': 7, 'b': 3.14}
    comm.send(data, dest=1)
elif
    rank == 1: data = comm.recv(source=0)
    print('On process 1, data is ', data)
```

Collective Communication

Broadcasting:

- Broadcasting takes a variable and sends an exact copy of it to all processes on a communicator.

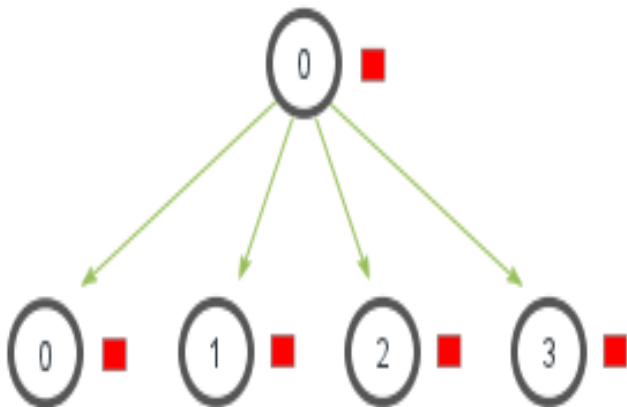

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
if rank == 0:
    data = {'key1': [1,2, 3], 'key2': ( 'abc', 'xyz')}
else:
    data = None
    data = comm.bcast(data, root=0)
print('Rank: ',rank,', data: ',data)
```

```
from mpi4py import MPI
import numpy as np
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
if rank == 0:
    # create a data array on process 0
    # in real code, this section might
    # read in data parameters from a file
    numData = 10
    data = np.linspace(0.0,3.14,numData)
else:
    numData = None
    # broadcast numData and allocate array on other ranks:
    numData = comm.bcast(numData, root=0)
if rank != 0:
    data = np.empty(numData, dtype='d')
    comm.Bcast(data, root=0)
    # broadcast the array from rank 0 to all others
print('Rank: ',rank, ', data received: ',data)
```

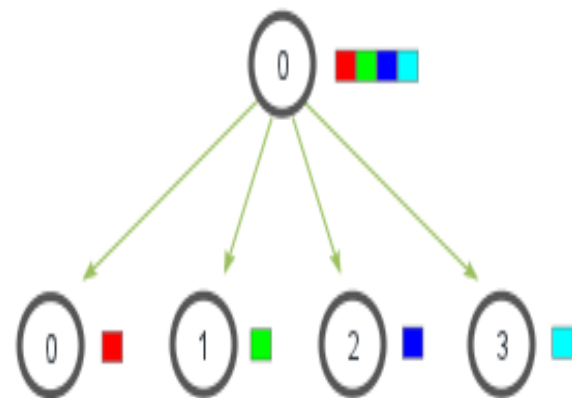
Scattering:

- Scatter takes an array and distributes contiguous sections of it across the ranks of a communicator

MPI_Bcast



MPI_Scatter

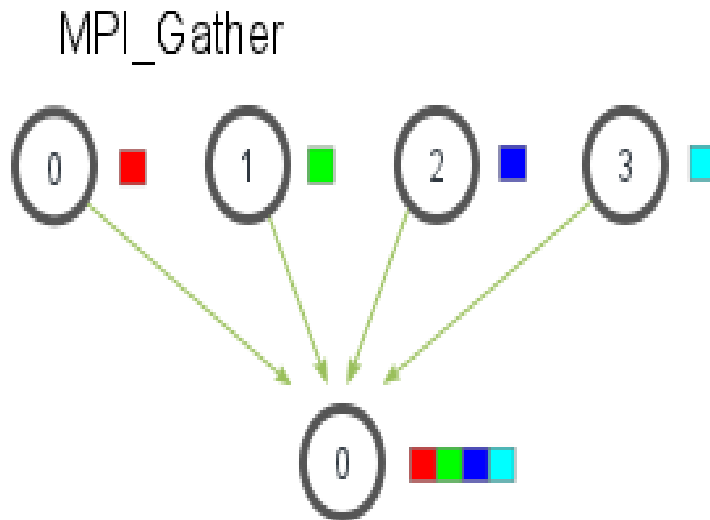


Example

```
from mpi4py import MPI
import numpy as np
comm = MPI.COMM_WORLD
size = comm.Get_size()
    # new: gives number of ranks in comm
rank = comm.Get_rank()
numDataPerRank = 10
data = None
if rank == 0:
    data = np.linspace(1, size*numDataPerRank, numDataPerRank*size)
    # when size=4 (using -n 4), data = [1.0:40.0]
recvbuf = np.empty(numDataPerRank, dtype='d')
    # allocate space for recvbuf
comm.Scatter(data, recvbuf, root=0)
print('Rank: ', rank, ', recvbuf received: ', recvbuf)
```

Gathering:

The reverse of a **scatter** is a **gather**, which takes subsets of an array that are distributed across the ranks, and *gathers* them back into the full array



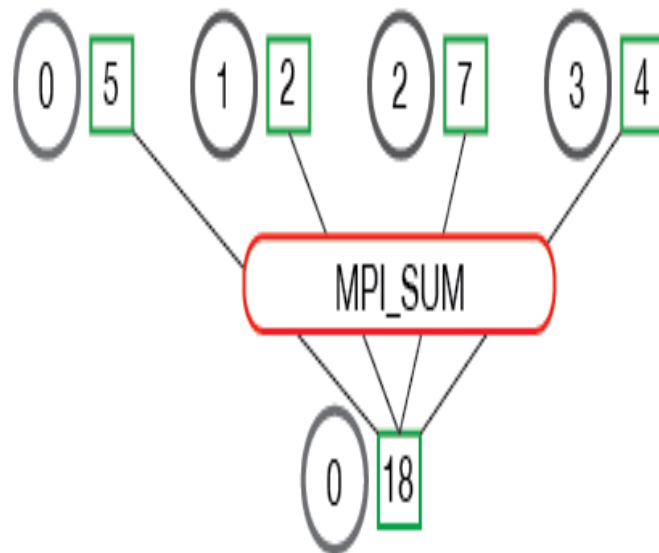
Example

```
from mpi4py import MPI
import numpy as np
comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()
numDataPerRank = 10
Sendbuf=np.linspace(rank*numDataPerRank+1,(rank+1)*numDataPerRank,numDataPerRank)
print('Rank: ',rank, ', sendbuf: ',sendbuf)
recvbuf = None
if rank == 0:
    recvbuf = np.empty(numDataPerRank*size, dtype='d') comm.Gather(sendbuf,
recvbuf, root=0)
if rank == 0:
    print('Rank: ',rank, ', recvbuf received: ',recvbuf)
```

Reduction:

- The MP **reduce** operation takes values in from an array on each process and reduces them to a single result on the root process.
- This is essentially like having a somewhat complicated send command from each process to the root process, and then having the root process perform the reduction operation.

MPI_Reduce



MPI_Reduce

